Dimensionality Reduction Approaches to Insider Threat Detection: From Principal Components to Interpretable Sparse Features

by

Suramya Raj Angdembay

A Thesis

Submitted to the Honors College of

The University of Southern Mississippi

in Partial Fulfillment

of Honors Requirements

Month Year

Approved by:

Dr. Haiyan Tian, Degree

School of Mathematics and Natural Sciences

Thesis Co-Advisor Name, Degree

School of Your School

Director Name, Degree

School of Your School

Joyce Inman, Ph.D., Dean

Honors College

ABSTRACT

Insider-threat detection requires distinguishing unusual activity from malicious activity. This thesis studies that distinction through principal component analysis, language-model surprisal, and sparse representations of a model's internal states on the synthetic CERT release 4.2 dataset. PCA scores daily activity by its reconstruction error. A language model trained on benign users scores the same kind of activity by how difficult its textual description is to predict. Separating predictions about a user's profile from predictions about their behavior improves the language model's ranking of insiders without retraining. Sparse-feature analysis then examines where selected internal features activate and how their removal changes predictions. These features contribute to behavior prediction, but the measured response does not establish an attack-specific mechanism. The main conclusion is that improving an anomaly score and explaining the model's internal computation are separate achievements: excluding profile predictions improves ranking, while profile information can still influence the behavior predictions that remain.

Keywords: Insider Threat; Dimensionality Reduction; Principal Component Analysis (PCA); Surprisal; Large Language Models; Mechanistic Interpretability; Sparse Autoencoders; One-Class Anomaly Detection

DEDICATION

Your dedication text here.

ACKNOWLEDGMENTS

Your acknowledgments begin here. Recognize advisors, faculty, funding agencies, and institutional support.

Contents

ABSTRACT **2**

DEDICATION **3**

ACKNOWLEDGMENTS **4**

LIST OF ABBREVIATIONS **9**

1 Introduction **10**

1.1 Example 1st Level Heading 10

Example 2nd Level Heading 10

2 Principal Component Analysis **11**

2.1 Principal Components: Motivation and Formulation 11

2.2 Inverse PCA and Reconstruction Error 13

2.3 A Worked Example and Its Geometry 14

2.4 Higher Dimensions and Anomaly Scoring 15

2.5 The CERT Dataset and Insider Cases 20

2.6 PCA Experiment: Preparation and Results 22

2.7 Summary 24

3 Language-Model Surprisal and Its Decomposition **26**

3.1 From Reconstruction to Prediction 26

3.2 Data Preparation and Text Representation 27

3.3 Next-Token Probability and Surprisal 30
3.4 Learning From Benign Days Only 32
3.5 Anomaly Scores and Their Decomposition 33
3.6 Evaluation and Corrected Results 35

4 Interpretable Dimensionality Reduction of the Fine-Tuned Model 39

4.1 Overview and Motivation 39
4.2 Hidden States and the Adaptation Difference 39
4.3 A Sparse Dictionary and a Worked Example 40
4.4 Choosing the Features to Test 41
4.5 Where Features Activate 42
4.6 Causal Test: Removing Features and Re-Scoring 43
4.7 Corrected Mechanistic Results 44
4.8 Limitations and Conclusions 45

REFERENCES 47

List of Tables

2.1 Rank-1 reconstruction of the four observations. 14

2.2 The three insider scenarios of CERT release 4.2, following the answer key. . . 21

2.3 CERT feature groups and training variability 23

2.4 PCA results for all 70 insiders 24

3.1 User-disjoint splits of CERT release 4.2: no user belongs to more than one group. 29

3.2 Class means and scores of the two example days. 35

3.3 Corrected language-model score decomposition 37

4.1 Sparse-feature activation location 43

4.2 Corrected feature-removal effects 44

List of Figures

2.1 Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance. 15

2.2 Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane. 16

2.3 A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data. 17

2.4 Synthetic two-dimensional dataset with principal component directions drawn from the center of the data. 18

2.5 Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates. 19

2.6 Values of the first principal component for all observations. 19

2.7 Values of the second principal component for all observations. 20

2.8 PCA ROC on all 70 insiders and 79 held-out benign users, using the 309 varying features. The marked threshold flags 35 insiders and 8 benign users. The curve is recomputed from the saved user scores of the original PCA run; no PCA refit is involved. 25

LIST OF ABBREVIATIONS

APA	American Psychological Association
AUC	Area Under the (Receiver Operating Characteristic) Curve
CERT	CERT Division of the Software Engineering Institute
LoRA	Low-Rank Adaptation
PCA	Principal Component Analysis
QLoRA	Quantized Low-Rank Adaptation
ROC	Receiver Operating Characteristic
SAE	Sparse Autoencoder
USM	The University of Southern Mississippi

Chapter I

Introduction

Begin the first chapter here. Indent the first line of each paragraph.

1.1 Example 1st Level Heading

Text for first-level heading.

Example 2nd Level Heading

Text for second-level heading.

Example 3rd Level Heading (In Paragraph)

Example 3rd Level Heading. Continue paragraph text immediately after the bold heading.

Chapter II

Principal Component Analysis

2.1 Principal Components: Motivation and Formulation

Principal Component Analysis (PCA) is a classical technique for identifying orthogonal directions along which the data exhibits the greatest variance. In applications such as behavioral cybersecurity logs, the raw feature space can be high-dimensional and strongly correlated, which complicates visualization, statistical modeling, and anomaly detection. PCA addresses these issues by constructing an orthonormal coordinate system whose axes (the *principal components*) are ordered by the amount of variance they explain. This chapter presents the mathematical formulation of PCA in the general p-dimensional setting, the inverse transformation that reconstructs data from the reduced space, and the reconstruction error that motivates PCA-based anomaly scoring, followed by low- and high-dimensional illustrations. Historically, the geometric origins of PCA trace back to Pearson, while the familiar algebraic eigenvalue formulation was developed by Hotelling [1, 2, 3].

PCA represents each observation as a numerical vector in $\mathbb{R}^p$ and approximates the data with a linear subspace. Its variance criterion summarizes dominant variation; it does not guarantee that high-variance directions are the most useful ones for detecting anomalies. Euclidean distance must also be meaningful in the chosen feature space. Because PCA is sensitive to the relative magnitude of features, the features are standardized first.

Let $X \in \mathbb{R}^{n \times p}$ be a data matrix with n observations and p features, whose ith row is the observation $\mathbf{x}_i^\top$. With the sample mean $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$ and the vector of ones $\mathbf{1} \in \mathbb{R}^n$, the centered data matrix is

$$X_c \;=\; X - \mathbf{1}\bar{\mathbf{x}}^\top,$$

which subtracts the mean from every observation. The sample covariance matrix is

$$S \;=\; \frac{1}{n-1} X_c^\top X_c \in \mathbb{R}^{p \times p}, \tag{2.1}$$

which is symmetric and positive semidefinite and therefore has an orthonormal eigendecomposition.

PCA looks for the unit direction $\mathbf{a}$ along which the projected scores $z_i = \mathbf{a}^\top(\mathbf{x}_i - \bar{\mathbf{x}})$ vary most. Their variance is $\mathbf{a}^\top S \mathbf{a}$, so the first principal direction solves

$$\max_{\mathbf{a} \in \mathbb{R}^p} \; \mathbf{a}^\top S \mathbf{a} \quad \text{subject to} \quad \mathbf{a}^\top \mathbf{a} = 1. \tag{2.2}$$

With a Lagrange multiplier λ, setting the gradient of $\mathbf{a}^\top S \mathbf{a} - \lambda(\mathbf{a}^\top \mathbf{a} - 1)$ to zero gives

$$S\mathbf{a} = \lambda \mathbf{a}. \tag{2.3}$$

The principal directions are therefore eigenvectors of S, and each eigenvalue is the variance explained along its direction [2, 3]. Ordering the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$ with orthonormal eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_p$ gives $S = U \Lambda U^\top$, with $U = [\mathbf{u}_1 \;\; \cdots \;\; \mathbf{u}_p]$ and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_p)$.

To reduce the dimension to $q < p$, keep the first q directions, $U_q = [\mathbf{u}_1 \;\; \cdots \;\; \mathbf{u}_q] \in \mathbb{R}^{p \times q}$. The reduced data, or *principal component scores*, are

$$Z \;=\; X_c U_q \in \mathbb{R}^{n \times q}, \tag{2.4}$$

whose ith row is $\mathbf{z}_i^\top = (U_q^\top \mathbf{x}_{c,i})^\top$. Since the total variance is $\mathrm{tr}(S) = \sum_{j=1}^{p} \lambda_j$, the proportion of variance explained by the first q components is

$$\mathrm{PVE}(q) \;=\; \frac{\sum_{j=1}^{q} \lambda_j}{\sum_{j=1}^{p} \lambda_j}, \tag{2.5}$$

and q is commonly chosen so that $\mathrm{PVE}(q)$ exceeds a fixed threshold such as 0.95.

2.2 Inverse PCA and Reconstruction Error

PCA provides dimensionality reduction, and the *inverse PCA transformation* approximates the original data from the reduced space:

$$\widehat{X} \;=\; Z\,U_q^\top + \mathbf{1}\bar{\mathbf{x}}^\top, \qquad \widehat{\mathbf{x}}_i \;=\; U_q \mathbf{z}_i + \bar{\mathbf{x}}, \tag{2.6}$$

where Z is the reduced data, U_q holds the principal components and $\bar{\mathbf{x}}$ is the mean vector. For the centered observation $\mathbf{x}_{c,i} = \mathbf{x}_i - \bar{\mathbf{x}}$ this is $\widehat{\mathbf{x}}_{c,i} = U_q U_q^\top \mathbf{x}_{c,i}$, the orthogonal projection onto the principal subspace $\mathrm{span}\{\mathbf{u}_1, \ldots, \mathbf{u}_q\}$.

The reconstruction error measures the difference between the original and the reconstructed data. For one observation it is

$$e_i \;=\; \|\mathbf{x}_i - \widehat{\mathbf{x}}_i\|_2^2 \;=\; \|(I - U_q U_q^\top)\,\mathbf{x}_{c,i}\|_2^2, \tag{2.7}$$

the squared Euclidean distance from $\mathbf{x}_{c,i}$ to the principal subspace, and over n observations it is the mean squared error

$$\mathrm{MSE} \;=\; \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{x}_i - \widehat{\mathbf{x}}_i\|_2^2. \tag{2.8}$$

A model built from normal data reconstructs normal observations well, so a large reconstruction error may indicate an anomaly. This is the basis of the PCA anomaly score used in this thesis.

2.3 A Worked Example and Its Geometry

For an illustration in the original units, consider four observations of two features,

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 2 \\ 5 & 4 \end{bmatrix}, \qquad \bar{\mathbf{x}} = \begin{bmatrix} 3.5 \\ 2.25 \end{bmatrix}, \qquad S = \begin{bmatrix} 1.6667 & 1.5 \\ 1.5 & 1.5833 \end{bmatrix}.$$

The eigenvalues of S are $\lambda_1 \approx 3.125$ and $\lambda_2 \approx 0.125$, so the first principal component $\mathbf{u}_1 \approx (0.72,\ 0.69)$ explains about 96% of the variance. Keeping $q = 1$, the scores are $z \approx (-1.94,\ -0.53,\ 0.19,\ 2.28)$, and the inverse transformation $\widehat{\mathbf{x}}_i = z_i \mathbf{u}_1 + \bar{\mathbf{x}}$ gives the reconstructions and errors in Table 2.1.

Table 2.1: Rank-1 reconstruction of the four observations.

i	$\mathbf{x}_i$	$\widehat{\mathbf{x}}_i$	$e_i = \lVert \mathbf{x}_i - \widehat{\mathbf{x}}_i \rVert_2^2$
1	(2, 1)	(2.10, 0.91)	0.018
2	(3, 2)	(3.12, 1.88)	0.029
3	(4, 2)	(3.64, 2.38)	0.274
4	(5, 4)	(5.15, 3.82)	0.054

The third observation lies farthest from the line spanned by $\mathbf{u}_1$ and has the largest error, and $\text{MSE} = (0.018 + 0.029 + 0.274 + 0.054)/4 \approx 0.094$. The same operations apply unchanged in high dimensions.

In $\mathbb{R}^2$, PCA identifies a new pair of orthonormal axes (u_1, u_2) obtained by rotating the original coordinate system so that u_1 aligns with the direction of maximum spread. If the data are approximately distributed along a line, reducing from 2D to 1D projects the points onto the line spanned by u_1 and discards the orthogonal component (Figure 2.1). In $\mathbb{R}^3$, the

three directions (u_1, u_2, u_3) are ordered by $(\lambda_1, \lambda_2, \lambda_3)$, and reducing to 2D projects onto the plane spanned by (u_1, u_2) (Figure 2.2).

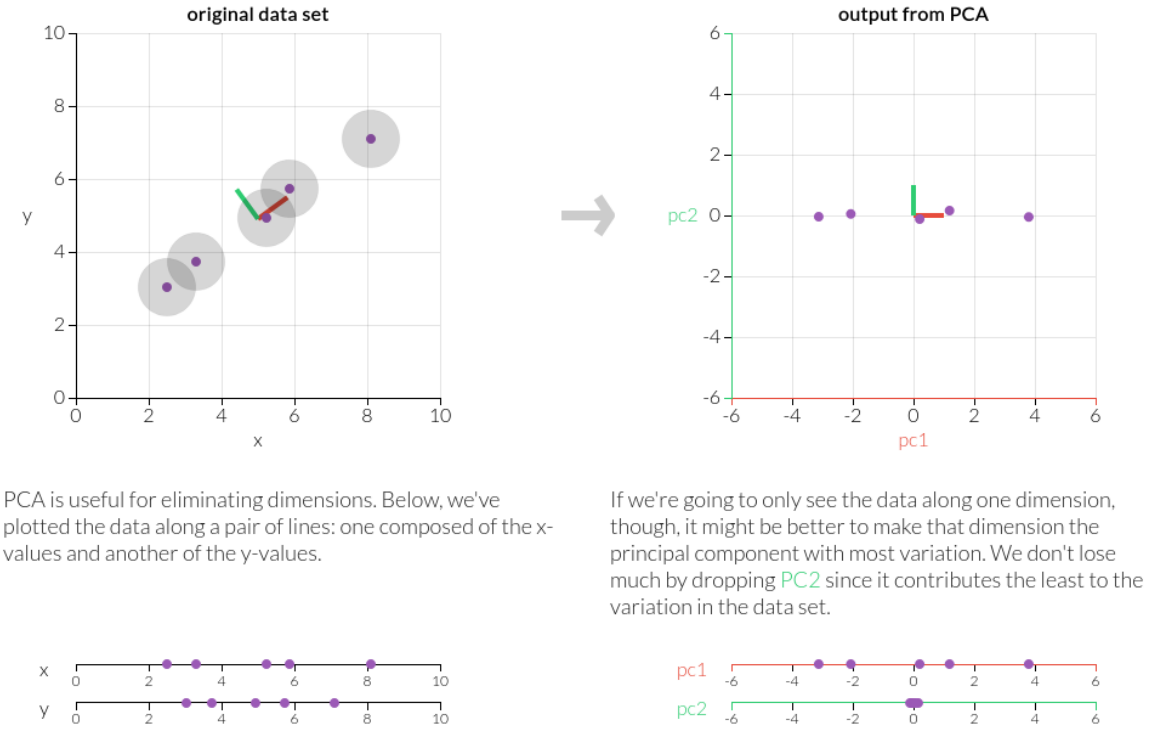


Figure 2.1: Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance.

2.4 Higher Dimensions and Anomaly Scoring

PCA is often most valuable when p is large and direct visualization is not possible. To illustrate this, consider a dataset where each observation corresponds to a country and each feature corresponds to a measured quantity, producing points in $\mathbb{R}^{17}$. For example, one may treat the weekly per-person consumption of 17 food categories as a vector in $\mathbb{R}^{17}$. Although we cannot visualize $\mathbb{R}^{17}$ directly, PCA provides low-dimensional views (e.g., $q = 1$ or $q = 2$) that reveal dominant contrasts among countries.

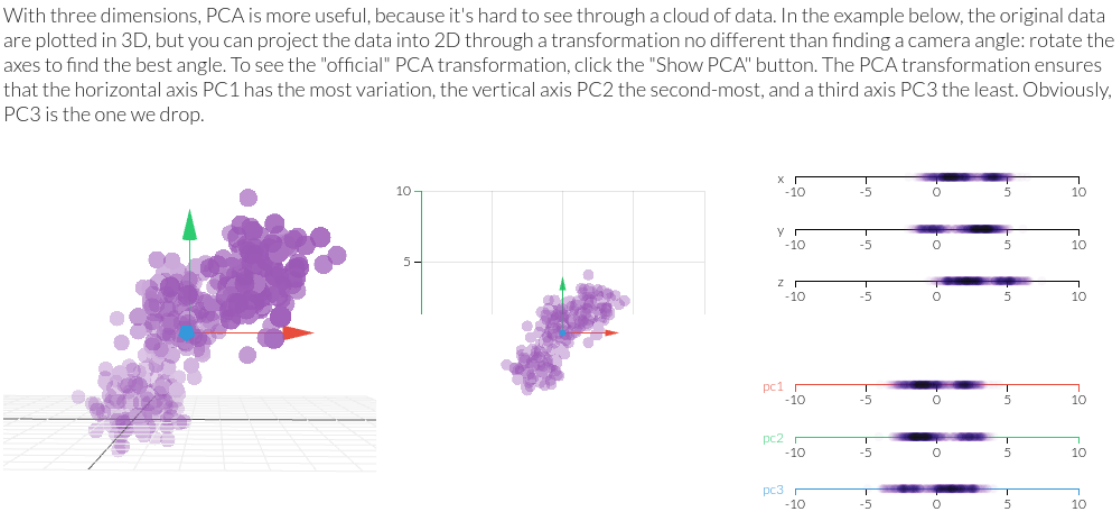


Figure 2.2: Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane.

Let $x_k \in \mathbb{R}^{17}$ denote the centered consumption vector for country k. The first PC score is $z_{k1} = u_1^\top x_k$, and the second is $z_{k2} = u_2^\top x_k$. Countries separated strongly along PC1 exhibit the greatest differences along the dominant consumption pattern, while PC2 captures the second most important pattern not explained by PC1.

A second illustration connects the geometry to anomaly scoring. To visualize how PCA organizes behavioral observations and highlights deviations from dominant patterns, we construct 30 two-dimensional points: 27 typical observations concentrated around a common trend, and three placed away from it to represent anomalous behavior. Figure 2.4 shows the data with the two principal directions drawn from its center; the first component aligns with the main trend and the second captures variation orthogonal to it. Projecting every observation onto these components gives the representation in Figure 2.5, and Figures 2.6 and 2.7 plot each observation's coordinate along the first and second components. Points that deviate from the general pattern become more clearly distinguishable in their component coordinates.

For a one-component reconstruction, an observation's error is the square of its discarded

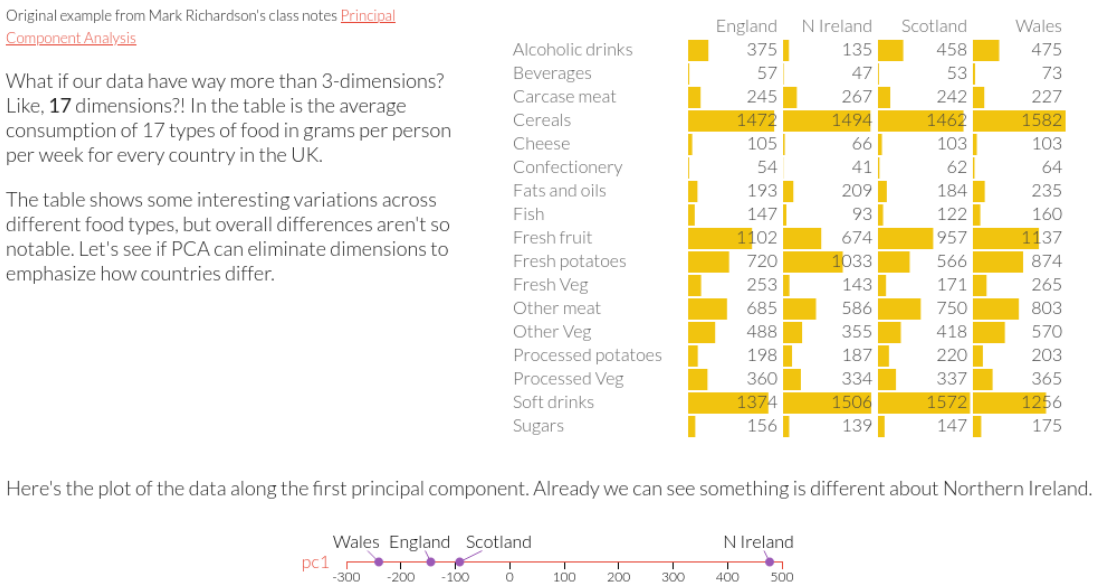


Figure 2.3: A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data.

PC2 coordinate. A large PC1 coordinate alone is therefore not evidence of a large reconstruction error. These examples illustrate geometry; the CERT experiment below fits its components on benign training users only.

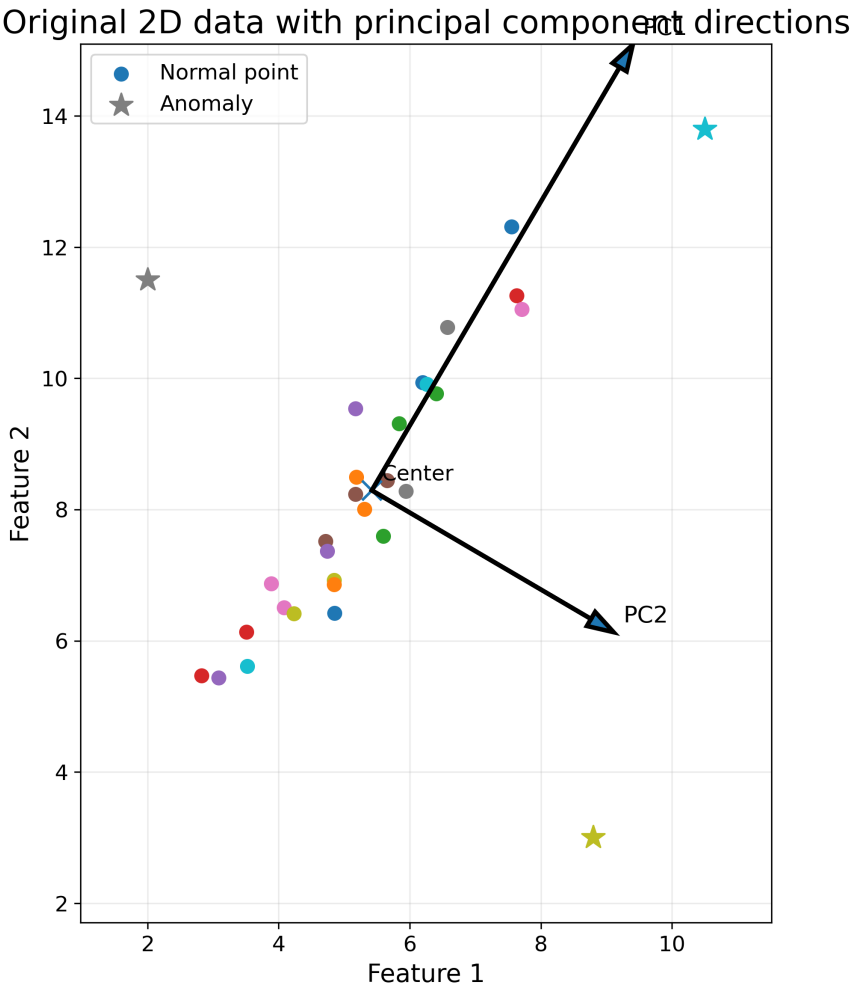


Figure 2.4: Synthetic two-dimensional dataset with principal component directions drawn from the center of the data.

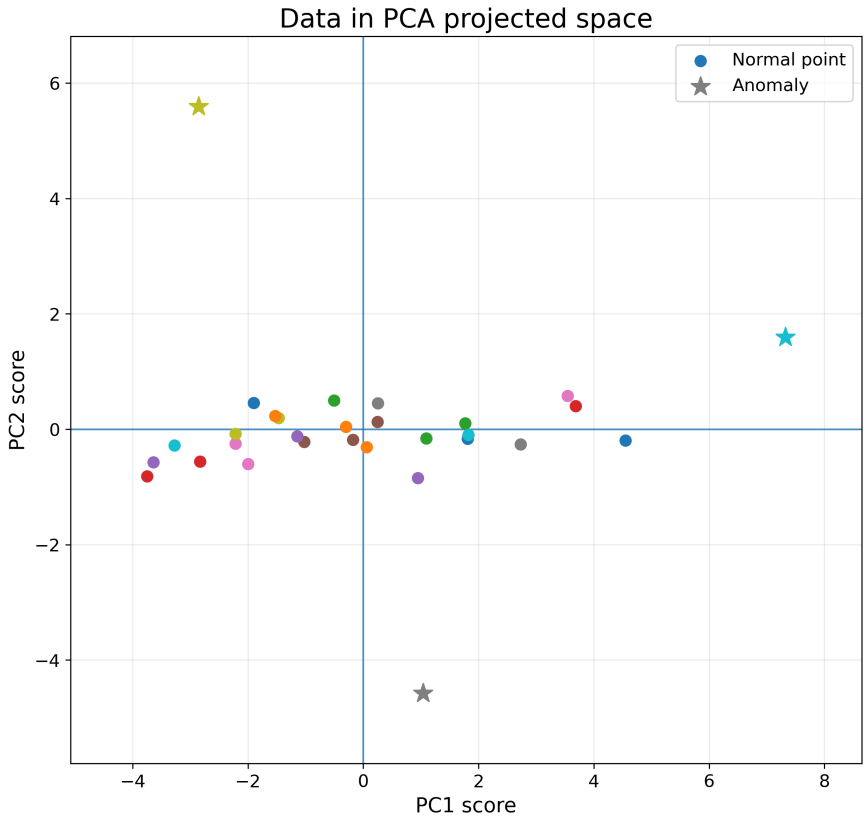


Figure 2.5: Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates.

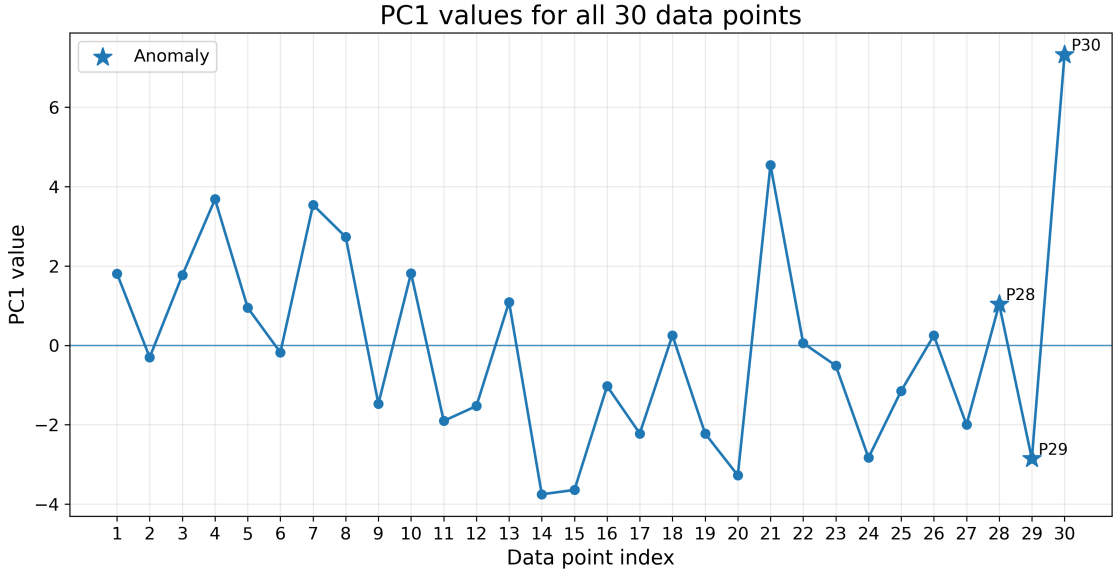


Figure 2.6: Values of the first principal component for all observations.

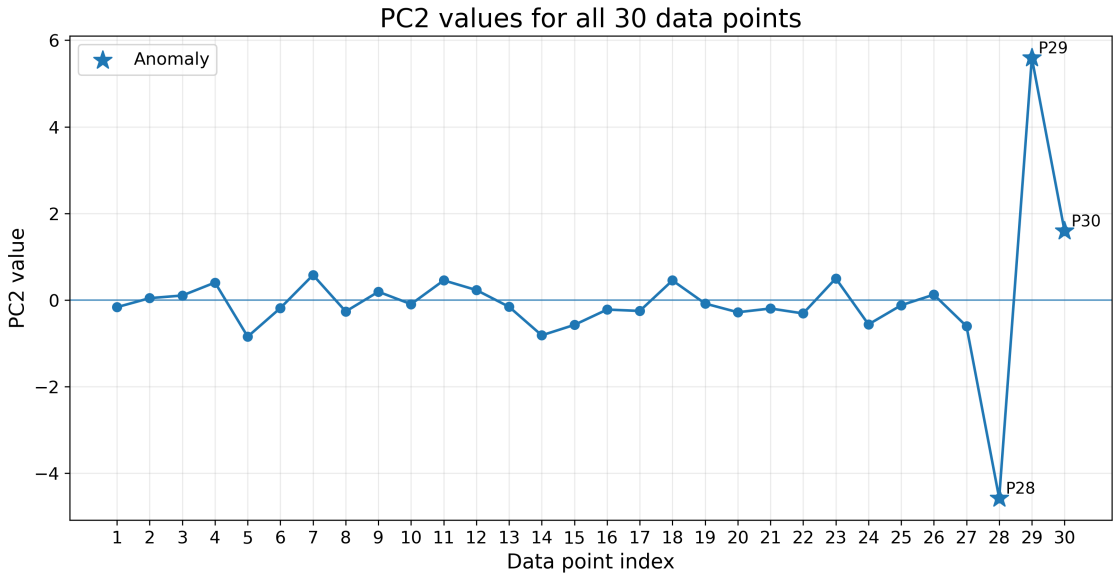


Figure 2.7: Values of the second principal component for all observations.

2.5 The CERT Dataset and Insider Cases

The CERT Insider Threat Test Dataset is a collection of synthetic datasets produced by the CERT Division of Carnegie Mellon University's Software Engineering Institute, with ExactData, LLC, under sponsorship from DARPA [5, 4]. Each dataset simulates an organization in which ordinary employees go about their work and a few simulated insiders act out a scripted malicious scenario. This thesis uses release 4.2, which simulates 1,000 employees whose recorded activity runs from 2 January 2010 to 17 May 2011. The data contain activity logs (logons and logoffs, removable-device connections, file operations, emails and web visits), organizational records (role, business unit, functional unit, department, team, and whether the employee is an IT administrator), and five personality scores for every employee.

Every log entry is an event $e = (u, \tau, s, \mathbf{a})$: a user $u \in \mathcal{U}$, a time τ, a source s (logon, device, file, email or web) and a list of attributes $\mathbf{a}$, with $|\mathcal{U}| = 1{,}000$. The unit of observation is the *user-day* $D = (u, t)$, one user on one calendar day, whose events are $E_{u,t} = \{e : e \text{ is an event of } u \text{ on day } t\}$. Every method in this thesis maps a user-day to an anomaly score $s(D) \in \mathbb{R}$, and a larger score means a more unusual day.

The dataset's answer key names each simulated insider, the scenario that insider follows, and the malicious events involved [4]. Release 4.2 contains 70 insiders in three scenarios (Table 2.2).

Table 2.2: The three insider scenarios of CERT release 4.2, following the answer key.

scenario	what the insider does	insiders
1	Starts working after hours and using a removable drive, which the user had not done before; uploads data to wikileaks.org and soon leaves.	30
2	Browses job websites and seeks work at a competitor; before leaving, copies data with a thumb drive at a much higher rate than before.	30
3	A disgruntled system administrator installs a keylogger on the supervisor's computer, logs in as the supervisor with the captured credentials and sends an alarming mass email, then leaves.	10

The language-model experiment uses the official incident window of each insider. If I_t is the calendar interval of day t, and $[a_u, b_u)$ is the answer key's incident interval, then

$$y(u,t) = \mathbb{I}[I_t \cap [a_u, b_u) \neq \varnothing] \tag{2.9}$$

for an official insider, and $y(u,t) = 0$ for other users. This identifies an *incident-window day*; it does not label every action on that day as malicious. An insider's other days are called *window-negative*, because absence from the window does not verify that all activity was benign. The calendar join uses December 28, 2009 as the day-index origin. Correct alignment gives 1,362 incident-window days from all 70 insiders among the 330,295 days with extracted sessions; another 530 calendar-window days have no session and are outside

that observation domain.

The PCA experiment uses a separate day-level feature table and its event-day flag: 966 days contain recorded malicious events (85, 861 and 20 by scenario). This is a different target from incident-window membership. Both experiments identify the same 70 official insiders, but their daily records and inputs differ, so their day-level metrics are not interchangeable.

Incident-window days account for about 0.41% of all session user-days. Calling every day negative would therefore give high accuracy while detecting no incidents. We instead evaluate rankings. A threshold on an anomaly score determines a true positive rate, the fraction of positive cases flagged, and a false positive rate, the fraction of negative cases flagged. The receiver operating characteristic (ROC) curve plots these rates as the threshold varies. Its area (AUC) is the probability that a positive case outranks a negative case, giving half credit for a tie. User-level AUC compares official insiders with non-insiders after taking each user's maximum daily score. A value of 1 is perfect ranking and 0.5 is chance-level ranking. This does not by itself measure which day an incident occurred.

2.6 PCA Experiment: Preparation and Results

The method above needs each observation as a vector of p numerical features, so the activity of each observation is summarized into one row $\mathbf{x}_i^\top$ of a matrix $X \in \mathbb{R}^{n \times p}$. Here an observation is a user-day, described by the day-level features of Le et al. [6] (Table 2.3). Release 4.2 gives $n = 330{,}452$ user-days with 502 features once identifiers and time indices are removed. Because PCA is sensitive to scale, each column is standardized with the mean and standard deviation of the training rows. Standardization needs a positive standard deviation, so the 193 columns that never vary among the training days are removed, leaving $p = 309$ features, or 299 without the organizational and personality attributes, the PCA counterpart of the behavior-only score of Chapter III.

PCA is fit on the 287,961 days of the 851 training users of Section 3.2, the users that also

Table 2.3: The day-level features of CERT release 4.2 [6], and how many of them vary among the training days. Activity features are also split by working hours and after hours and by the kind of computer used (the user's own, a shared one, another user's or the supervisor's). Files are split into six types and websites into six kinds, among them job-search, cloud-storage, leak and hacking websites. The profile attribute that does not vary is the business unit, which takes a single value in this release.

group	what is measured	features	varying
all activity	events of any kind	15	14
logons	logons	15	14
removable drives	connections and their mean duration	18	14
files	accesses by type, with mean length, depth and word count	210	132
emails	recipients, attachments, size and text length	42	33
web	visits by kind of website, with mean URL and content length	189	90
calendar	weekday or weekend	2	2
profile	organizational attributes and five personality scores	11	10
total		502	309

train the language model; all of them are benign, so PCA sees no insider and no held-out benign user. It keeps the first q principal directions U_q, with q the smallest number whose proportion of variance explained (2.5) reaches 95%; this gives $q = 71$ directions, which explain 95.1% of the variance ($q = 63$ without the profile attributes). Each observation is reduced to its scores and mapped back to the feature space with the inverse PCA transformation (2.6), and its anomaly score is its reconstruction error e_i (2.7); the mean squared error (2.8) summarizes a set of observations. Benign behavior is reconstructed well by the dominant directions of benign variation, so a large e_i marks an observation that those directions cannot explain.

The median reconstruction error is 6.6 on the days of the 79 held-out benign users and 17.0 on the 966 malicious days, and 6.4% of the malicious days exceed the 99th percentile of the training errors, against 0.5% of the held-out benign days. Scoring each user by their largest error gives the user-level AUCs of Table 2.4 and the ROC curve of Figure 2.8. For

all 70 insiders, the user AUC is 0.777. Removing profile attributes changes it little. The methods use different representations and sets of observed days; this is not a controlled comparison of model architectures. The last PCA row shows why the constant columns are removed. They include visits to leak and hacking websites and logons to the supervisor's computer, which no training day contains and which occur, in the evaluation data, almost only on malicious days. Left in their raw units, as some software does for a column with zero standard deviation, they carry 99.7% of the malicious days' squared error.

Table 2.4: PCA user-level AUC for all 70 official insiders against 79 held-out benign users, using each user's largest daily reconstruction error. Intervals resample both insider and benign users, with 10,000 draws. All rows retain 95% of training variance.

PCA input	p	q	user AUC [95% interval]
Varying features	309	71	0.777 [0.701, 0.848]
Varying behavior features	299	63	0.791 [0.716, 0.860]
All features, constants in raw units	502	71	0.832 [0.764, 0.894]

2.7 Summary

PCA describes high-dimensional observations through a small set of orthogonal directions. The discarded component gives a reconstruction error that can serve as an anomaly score. The CERT example also shows that preprocessing matters: retaining training-constant columns in raw units can make a few rare event fields dominate that error. The next chapter considers a different model of normal activity and asks which parts of its score help distinguish insiders.

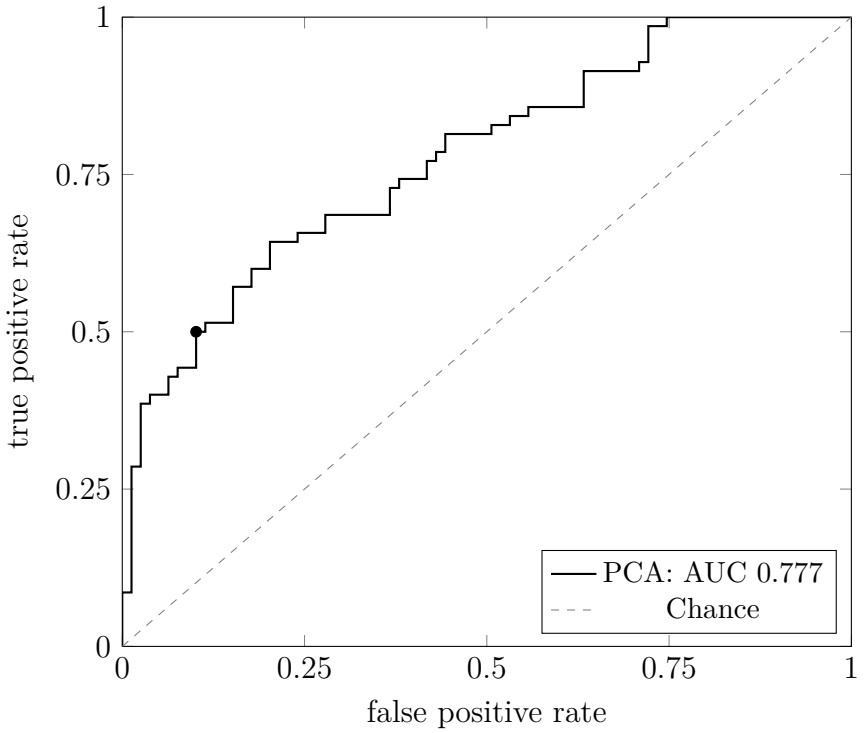


Figure 2.8: PCA ROC on all 70 insiders and 79 held-out benign users, using the 309 varying features. The marked threshold flags 35 insiders and 8 benign users. The curve is recomputed from the saved user scores of the original PCA run; no PCA refit is involved.

Chapter III

Language-Model Surprisal and Its Decomposition

3.1 From Reconstruction to Prediction

Chapter II scored an observation by the reconstruction error of a low-dimensional linear model. This chapter scores a user-day, one user's activity on one day, by how difficult its text record is for a large language model, fine-tuned (further trained) only on benign activity, to predict. Both scores ask the same question: how badly does a model of normal behavior account for this observation? Two ideas from Chapter II carry over. The fine-tuning changes each adapted weight matrix only by a low-rank correction (Section 3.4), and Chapter IV decomposes the model's internal change with a sparse relative of PCA. Scoring activity by how poorly a model trained on normal activity predicts it follows earlier work on log and insider-threat data [10, 11].

The new contribution of this chapter is to take the score apart. We show that it splits exactly into a part describing *who* the user is and a part describing *what* the user did, and examine how counting those predictions changes the ranking of insiders.

The approach starts with a *pretrained* model that has already learned to predict text. We adapt it to records from benign employees, then keep its parameters fixed during evaluation. Each day's record is broken into small text pieces called *tokens*. Given the actual preceding tokens, the model assigns a probability to the next observed token. Low probabilities give larger scores, which we average over the day. This measures how unusual the record is under the model; it is not a probability that the employee is malicious. Finally, we compare the

score for all tokens with a score that counts only predictions about behavior, while leaving the input unchanged.

3.2 Data Preparation and Text Representation

We use CERT release 4.2, a synthetic dataset of employee activity containing ordinary work and scripted insider incidents. The language model reads activity grouped into *sessions*, each running from a logon to the matching logoff on one computer, with features following the extraction of Le et al. [6]: time of day, duration, computer, and counts of logon, device, file, email and web events. We write these measurements as text so that the language model can read them. For example, a session lasting 489 minutes with 3 emails is represented by fields such as `duration=489` and `n_email=3`. The model receives these activity summaries, rather than the employees' email messages.

Each user-day D is written as the text

$$T(D) \;=\; \texttt{DAY} \,\|\, \texttt{PSY} \,\|\, \texttt{SESSIONS} \,\|\, \texttt{SES}_1 \,\|\, \cdots \,\|\, \texttt{SES}_k,$$

lines joined by newlines: the organizational attributes (`DAY`), the personality scores (`PSY`), the number of sessions (`SESSIONS`) and one line of `key=value` features per session; $\|$ denotes joining the lines. Days have between one and nine sessions (two thirds have one), and the longest texts have about 1,300 tokens, within the model's limit.

The record below is a real benign day from CERT release 4.2 (user `AAF0535`, day 7). The session line is a single line in the actual text; it is wrapped here only to fit the page.

```
DAY week=1 project=na role=39 b_unit=0 f_unit=4 dept=15 team=11 itadmin=0
PSY O=17 C=21 E=36 A=33 N=31
SESSIONS total=1 kept=1
SES idx=0 pc=0 isworkhour=1 isafterhour=0 isweekend=0 isweekendafterhour=0
    duration=489 n_concurrent_sessions=1 start_with=1 end_with=1
```

```
ses_start=8.933 ses_end=17.083 n_allact=34 n_logon=1 n_usb=0
usb_mean_usb_dur=0 n_file=0 file_n-disk1=0 file_n_exef=0 n_email=3
email_mean_n_atts=1.333 n_http=29 http_n_jobf=0 http_n_cloudf=0
http_n_leakf=0 http_n_hackf=0
```

This user had one working-hours session on their usual computer, lasting 489 minutes, with 3 emails and 29 web requests. The lines fall into two classes. The **profile lines** (P), `DAY` and `PSY`, describe *who* the user is: organizational attributes and Big-Five personality scores, rather than the session activity counts. The **behavior lines** (B), `SESSIONS` and `SES`, describe *what the user did* that day. The profile group also contains the week field on the `DAY` line: this is a grouping by text location, not a claim that every field in it is an identity attribute.

The model reads *tokens* $x_1, \ldots, x_n$, short pieces of text in which numbers are split into single digits, so `dept=15` is the four tokens `dept`, `=`, `1`, `5`. A token's class is the class of the line it lies on,

$$c(x_t) \;=\; \begin{cases} P & \text{if } x_t \text{ lies on a } \texttt{DAY} \text{ or } \texttt{PSY} \text{ line,} \\ B & \text{otherwise (session count, session activity, or residual text),} \end{cases}$$

so the class is decided by where a token sits, not by what it is: the digit `1` is a profile token in `team=11` and a behavior token in `n_logon=1`. The record above has $N = 212$ scored tokens, of which $N_P = 54$ are profile tokens and $N_B = 158$ behavior tokens. The profile share varies between days. Each token's surprisal is counted in the class of the token being predicted, while the model still reads the whole preceding text as context: the split decides which predictions are *counted*, not what the model *sees*.

The program that converts measurements into text omitted four hyphenated file fields: `file_n-to_usb1`, `file_n-from_usb1`, `file_n-file_act3`, and `file_n-disk1`. If a missing field appears as a zero in the text, that zero is a serialization default, not a measured absence of activity. The results concern this recorded representation.

Release 4.2 supplies 470,611 sessions on 330,295 user-days. A day without an extracted session has no text and is outside this experiment. For evaluation, an *incident-window day* is a day overlapping an insider's recorded incident period. An insider's other days are *window-negative*; this does not verify that all their activity was benign. Section 2.5 gives the precise calendar rule. The corrected evaluation labels cover all 1,362 observed incident-window days from the 70 official insiders.

To evaluate employees excluded from adaptation, we split the data by user, keeping all days of a user together. The three groups are training users, held-out benign users, and insiders, written as $\mathcal{U}_{\text{train}}$, $\mathcal{U}_{\text{val}}$, and $\mathcal{U}_{\text{ins}}$. All official insiders belong to the last group. Among the other users, a fixed hash of each identifier assigns about one tenth to the held-out group and the rest to training; the hash makes the assignment repeatable (Table 3.1). *Held out* here means excluded from fine-tuning. The evaluation users have been examined during research development, so they are not a newly untouched test set.

Table 3.1: User-disjoint splits of CERT release 4.2: no user belongs to more than one group.

group	users	user-days	incident-window days
$\mathcal{U}_{\text{train}}$ (training)	851	287,827	0
$\mathcal{U}_{\text{val}}$ (held-out benign)	79	27,026	0
$\mathcal{U}_{\text{ins}}$ (insiders)	70	15,442	1,362

The evaluation pool contains every observed session-day of the 70 insiders and the 79 held-out benign users: 42,468 days from 149 users. The calendar-label correction changed evaluation labels and score coverage; the model and its benign training membership remain unchanged.

3.3 Next-Token Probability and Surprisal

A token that the model assigns probability 0.8 is less surprising than one assigned probability 0.01. We measure this difference with *surprisal*, the negative natural logarithm of the assigned probability: these examples give approximately 0.223 and 4.605. Lower probabilities therefore contribute more to the anomaly score.

We score the text that was actually observed. At position t, the model uses only the preceding tokens $x_{<t} = (x_1, \ldots, x_{t-1})$ to predict x_t. The evaluator knows the whole record, but the model's prediction at that position cannot use the target token or later tokens. This evaluates a recorded day's predictability; it does not generate a new activity record or forecast the next day.

The probabilities come from the model's internal representation of that preceding text. Its 36 processing layers produce a *context vector* $\mathbf{h}_{t-1} \in \mathbb{R}^{4096}$, a list of numbers summarizing the text for the next prediction [12]. Each possible token v in the vocabulary $\mathcal{V}$ has a learned vector $\mathbf{w}_v$. Their dot product gives a score, or *logit*, $o_v = \mathbf{w}_v^\top \mathbf{h}_{t-1}$; collecting these scores gives $\mathbf{o} = W_{\text{out}}\mathbf{h}_{t-1}$. The *softmax* converts the scores into probabilities by exponentiating each score and dividing by the sum over all possible tokens:

$$p_\theta(x_t = v \mid x_{<t}) \;=\; \frac{e^{o_v}}{\sum_{v' \in \mathcal{V}} e^{o_{v'}}}, \tag{3.1}$$

where θ denotes the model's learned parameters. For finite logits, these probabilities are positive and sum to 1. Only differences between logits matter: for two tokens a and b, $p(a)/p(b) = e^{o_a - o_b}$.

The *surprisal* of the actual token is

$$\ell_t \;=\; -\log p_\theta(x_t \mid x_{<t}) \;=\; \mathrm{LSE}(\mathbf{o}) \;-\; o_{x_t}, \qquad \mathrm{LSE}(\mathbf{o}) = \log \sum_v e^{o_v}, \tag{3.2}$$

where the log-sum-exp satisfies $o_{\max} \le \mathrm{LSE}(\mathbf{o}) \le o_{\max} + \log|\mathcal{V}|$. The log-sum-exp term com-

bines the scores of all possible tokens into one value. Subtracting the observed token's logit gives its surprisal. Thus the measure depends on the probability assigned to the observed token, including how strongly other tokens compete with it.

Why use the negative logarithm? Besides assigning larger values to rarer events, it turns a product of probabilities into a sum of scores. This lets us add the surprises of successive predictions. More formally, two requirements on a measure of surprise $S(p)$ leave only the logarithm, the measure of information introduced by Shannon [18]. First, rarer events are more surprising, so S decreases in p. Second, the surprises of independent events add, and since $P(A \text{ and } B) = P(A)P(B)$ this means

$$S(pq) \;=\; S(p) + S(q) \qquad \text{for all } p, q \in (0, 1]. \tag{3.3}$$

Writing $p = e^{-x}$ and $g(x) = S(e^{-x})$ turns (3.3) into $g(x+y) = g(x)+g(y)$: adding two inputs adds their outputs. This is Cauchy's functional equation, whose only increasing solutions are $g(x) = c\,x$. Hence

$$S(p) \;=\; -c \ln p, \qquad c > 0, \tag{3.4}$$

where c only fixes the unit; using the natural logarithm with $c = 1$ measures surprise in *nats*. A certain event carries no surprise, an event thought impossible would be infinitely surprising, and every tenfold drop in probability adds the same $\ln 10$ nats.

For a single observed event, the surprisal is also its *negative log-likelihood*. By the chain rule of probability, the probability of a day's text is the product of each token's probability given the tokens before it, so

$$-\log p_\theta(x_2, \ldots, x_n \mid x_1) \;=\; \sum_{t=2}^{n} -\log p_\theta(x_t \mid x_{<t}) \;=\; \sum_{t=2}^{n} \ell_t : \tag{3.5}$$

the total surprisal of a day is the negative log-likelihood of its text. This chain rule uses conditional probabilities and does not assume that successive tokens are independent. Because

days differ in length, the anomaly score of Section 3.5 divides this total by the number of scored tokens.

3.4 Learning From Benign Days Only

Our starting model is Qwen3-8B [15], a language model already trained on a large text corpus. *Fine-tuning*, also called adaptation, is further training that makes it better at predicting our activity records. Only this further training is restricted to benign employees. Once adaptation finishes, we freeze the model: scoring evaluation records does not update its parameters.

During adaptation, the model predicts each training token from its preceding text. The training objective adds up the resulting surprisal, or prediction *loss*. Reducing this loss encourages the model to assign higher probabilities to the benign records it sees. Formally, we seek parameters that minimize

$$\theta^\star \;=\; \arg\min_{\theta} \sum_{D \in \mathcal{B}} \sum_{t=2}^{n_D} -\log p_\theta\big(x_t^{(D)} \mid x_{<t}^{(D)}\big), \tag{3.6}$$

where $\mathcal{B}$ is the set of benign training days and $x^{(D)}$ is the text of day D, with n_D tokens. This is *maximum-likelihood estimation*: choosing parameters that make the observed training text more probable. Iterative training seeks a low-loss solution; the notation does not claim that we find a global minimum.

A simple frequency model illustrates the idea. Suppose it assigns one probability p_j to each of K tokens, without considering preceding text. If token j occurs N_j times among N training tokens, minimizing $\sum_j N_j(-\log p_j)$ subject to $\sum_j p_j = 1$ with a Lagrange multiplier gives

$$\hat{p}_j \;=\; \frac{N_j}{N}, \tag{3.7}$$

the observed token frequencies. A token appearing in 80 of 100 observations receives proba-

bility 0.8 in this simple model. The language model instead learns probabilities that depend on the preceding text. It reduces its loss by *gradient descent*, repeatedly adjusting trainable parameters in a direction that lowers training loss. This motivates using surprisal to detect unusual records, but rarity alone does not establish malicious intent.

We restrict which parameters can change using *low-rank adaptation* (LoRA) [13]. The original weight matrices stay fixed, and selected matrices receive a trainable correction. For a square matrix, this has the form $W' = W + GH$, with $G \in \mathbb{R}^{d\times r}$, $H \in \mathbb{R}^{r\times d}$, and $r = 16$. The two smaller matrices provide the correction while using far fewer trainable parameters.[1] For any input $\mathbf{x}$, the change $GH\mathbf{x}$ lies in the span of the 16 columns of G. This gives a limited connection to PCA: each adapted matrix changes its output along at most 16 directions. The composed nonlinear network's hidden-state difference need not lie in a 16-dimensional subspace.

The quantized version, QLoRA [14], stores the fixed base weights in a 4-bit representation to reduce memory use, while learning the low-rank corrections. Our adaptation makes one pass over the 287,827 benign user-days from the 851 training users in Table 3.1. No insider records enter this training objective.

3.5 Anomaly Scores and Their Decomposition

The previous sections explain how the model assigns a score to each predicted token. To compare whole days, we average those scores. Averaging prevents a longer record from receiving a larger score simply because it contains more terms in the sum. Let $\mathcal{T}$ be the set of scored positions of a day (every token except the first, which has nothing before it) and

[1] For a 4096×4096 matrix, the two rank-16 factors contain 131,072 parameters, about 0.78% of the 16,777,216 entries in the original matrix. Rectangular weight matrices use the corresponding input and output dimensions.

$N = |\mathcal{T}|$. The anomaly score of the day is its average token surprisal,

$$\bar{\ell} \;=\; \frac{1}{N}\sum_{t\in\mathcal{T}} \ell_t. \tag{3.8}$$

A high $\bar{\ell}$ means that the model adapted on benign days found this day hard to predict. A user's score is the score of their most unusual day,

$$\bar{\ell}(u) \;=\; \max_{D\in\mathcal{D}_u} \bar{\ell}(D),$$

where $\mathcal{D}_u$ is the set of that user's observed days. This maximum asks whether the user has any particularly unusual day. It is a chosen summary rule, so users with longer observed histories have more opportunities to receive a high score.

The full average includes predictions about both the user's profile and their activity. A rare profile can therefore raise the score even when the activity is ordinary. We examine this contribution by adding the losses in each token class separately. Divide the scored positions into profile and behavior positions, $\mathcal{T} = \mathcal{T}_P \cup \mathcal{T}_B$, with $N = N_P + N_B$. Let $L_P = \sum_{t\in\mathcal{T}_P} \ell_t$ and $L_B = \sum_{t\in\mathcal{T}_B} \ell_t$ be their summed losses, and let $L = L_P + L_B$. For nonempty classes, the class means are $\bar{\ell}_P = L_P/N_P$ and $\bar{\ell}_B = L_B/N_B$. Then

$$\bar{\ell} \;=\; \frac{1}{N}\big(L_P + L_B\big) \;=\; \frac{N_P}{N}\,\bar{\ell}_P \;+\; \frac{N_B}{N}\,\bar{\ell}_B : \tag{3.9}$$

the day's score is exactly a weighted average of the profile mean and the behavior mean, weighted by their shares of the tokens. The behavior-only score

$$\bar{\ell}_B \;=\; \frac{L_B}{N_B} \;=\; \frac{L - L_P}{N - N_P} \tag{3.10}$$

uses the same model with no retraining. It adds only behavior-target losses and divides by the number of those targets. Profile text still precedes the activity text, so it can still

influence the predictions. A *target* here is the token being predicted.

To remove a class from an average, both its summed loss and its token count must be removed. Simply subtracting the profile mean gives a different quantity:

$$\bar{\ell} - \bar{\ell}_P \;=\; \frac{N_B}{N}\big(\bar{\ell}_B - \bar{\ell}_P\big)\,: \tag{3.11}$$

this measures how much harder the behavior was to predict than the profile, not how hard the behavior was to predict, and it can reverse the order of two days.

For a numerical illustration, two user-days each have five scored tokens, one profile token and four behavior tokens, so $N_P/N = 0.2$. Day A belongs to a malicious user with a common profile and unusual behavior; Day B to a benign user with a rare profile and routine behavior.

Table 3.2: Class means and scores of the two example days.

day	$\bar{\ell}_P$	$\bar{\ell}_B$	full score $\bar{\ell} = 0.2\,\bar{\ell}_P + 0.8\,\bar{\ell}_B$	behavior-only score
A (malicious)	0.1054	0.9324	0.7670	0.9324
B (benign)	2.9957	0.4125	0.9292	0.4125

The full score ranks the benign Day B above the malicious Day A: its profile part favors Day B by 0.578 and outweighs a behavior part that favors Day A by 0.416. The behavior-only score restores the correct order. The naive subtraction can fail as well: for a Day C with $\bar{\ell}_P = 3.0$, $\bar{\ell}_B = 1.2$ and a Day D with $\bar{\ell}_P = 0.2$, $\bar{\ell}_B = 0.8$, equation (3.11) gives $0.8(1.2 - 3.0) = -1.44$ and $0.8(0.8 - 0.2) = 0.48$, ranking D above C although C's behavior is less predictable.

3.6 Evaluation and Corrected Results

Evaluation asks whether larger scores belong to the cases we want to find. There are two questions: do insider *users* rank above benign users, and do incident-window *days* rank above

negative days? The first uses each user's maximum daily score; the second uses individual day scores. Success on one does not guarantee success on the other.

The receiver operating characteristic (ROC) curve records the fraction of positives found and the fraction of negatives flagged as a score threshold varies. Its area, ROC-AUC or simply AUC, also has a direct ranking interpretation: the share of positive–negative pairs in which the positive has the larger score, with half credit for ties. A value of 1 means perfect ranking; 0.5 is chance-level ranking. A value below 0.5 means that the observed ranking favors negatives more often than positives. Establishing whether a small departure from 0.5 is reliable requires a separate uncertainty assessment.

With one malicious user m compared against n_- benign users $u_1, \ldots, u_{n_-}$, the area under the ROC curve for m is the fraction of benign users that m outranks,

$$\mathrm{AUC}_m \;=\; \frac{1}{n_-}\sum_{j=1}^{n_-}\left(\mathbb{I}[\bar{\ell}(m) > \bar{\ell}(u_j)] + \tfrac{1}{2}\mathbb{I}[\bar{\ell}(m) = \bar{\ell}(u_j)]\right), \tag{3.12}$$

where $\mathbb{I}[\cdot]$ is 1 when the condition holds and 0 otherwise. Averaging over M malicious users gives $\mathrm{AUC} = \frac{1}{M}\sum_m \mathrm{AUC}_m$. Because every malicious user is compared with the same benign users, this average equals the pooled AUC over all $M \times n_-$ malicious–benign pairs:

$$\frac{1}{M}\sum_m \frac{1}{n_-}\sum_j w_{mj} \;=\; \frac{1}{M\,n_-}\sum_m\sum_j w_{mj},$$

where w_{mj} is 1, $\frac{1}{2}$, or 0 for a win, tie, or loss.

For example, suppose four benign users have full scores (0.93, 0.70, 0.85, 0.60) and a malicious user has full score 0.77. The malicious user outranks 0.70 and 0.60, so $\mathrm{AUC}_m = 2/4 = 0.50$. If the behavior-only scores are (0.41, 0.55, 0.70, 0.48) for the benign users and 0.93 for the malicious user, it outranks all four, and $\mathrm{AUC}_m = 4/4 = 1.00$.

For the day-level evaluation, we repeat the comparison once for each insider. Each comparison, called a *fold*, contains all observed days of that insider and all days of the same 79 held-out benign users. Incident-window days are positive; the insider's window-negative

days and all benign-user days are negative. We calculate each fold's metrics and average them equally across the 70 insiders. The model stays fixed across folds; we do not train a different model for each comparison. Unlike the user-AUC identity above, averaging day-metric folds need not equal pooling all days into one calculation.

We also report *average precision* (AP). At a given threshold, *precision* is the fraction of flagged cases that are positive, while *recall* is the fraction of all positive cases that are flagged. Lowering the threshold flags more cases and changes both fractions. AP summarizes precision over the increases in recall; it is not the precision at one chosen alert threshold. Table 3.3 compares the full, behavior-only and profile-only scores from the same frozen model and input.

Table 3.3: Corrected CERT r4.2 score decomposition: 42,468 days, including 1,362 incident-window days, from 70 insiders and 79 held-out benign users. All views retain the same input and model. AUC and AP columns are equal-insider fold means.

Scored targets	day AUC	day AP	user AUC
All tokens	0.540	0.004859	0.589
Behavior tokens	0.820	0.025238	0.825
Profile tokens	0.482	0.001353	0.476

The user AUC rises from 0.589 to 0.825 when we count only behavior targets. This means that, with half credit for ties, an insider ranks above a benign user in about 82.5% of the evaluated user pairs, compared with 58.9% for the full score. These are ranking results, not percentages of users correctly classified at an alert threshold.

To describe uncertainty in the change, we use a *user bootstrap*: repeatedly sample the 70 insiders with replacement and recalculate the mean gain, keeping each insider's full-score and behavior-only AUCs paired. Counting only behavior targets raises user AUC by 0.236, with a descriptive 95% user-bootstrap interval $[0.185, 0.289]$. This resampling keeps the model and benign comparison cohort fixed; it does not measure variation across independently trained models. The improvement comes from changing which prediction losses enter the score. The profile-only point estimate is near chance and slightly below it; that alone does not establish

a systematic reversed ranking. The paired score comparison is the direct evidence for the benefit of behavior-only scoring in this experiment.

AP depends on how common positives are in the comparison population. That fraction is called *prevalence*. Each fold includes only one insider alongside all 79 benign users, so its positive days are especially rare: mean fold prevalence is about 0.0714%. Pooling all 42,468 days instead includes every insider together and gives 3.21% positive days. The reported AP values average the individual folds and must be interpreted using that comparison, rather than the pooled fraction.

This result separates score composition from internal information use. Profile predictions can dilute the ranking even though the behavior predictions may depend on the preceding profile text. Excluding those profile predictions from an average does not remove that context. The result applies to this model, text representation and benchmark, whose evaluation users were repeatedly examined during development. Better user ranking also does not establish that the highest-scoring day is the day of the incident. The main conclusion is that changing the composition of an anomaly score can improve ranking without changing the model. Explaining how the model uses the information in its input requires a different test; the next chapter investigates that question by editing internal features.

Chapter IV

Interpretable Dimensionality Reduction of the Fine-Tuned Model

4.1 Overview and Motivation

Chapter III examined the score from the outside: which tokens its surprisal comes from. *Mechanistic interpretability* asks what the model computes internally [16, 17]. This chapter answers that question with a form of dimensionality reduction designed for interpretation. PCA (Chapter II) reduces data to a few dense directions, each of which mixes many original features. A *sparse autoencoder* instead describes each observation with a few directions drawn from a much larger dictionary; because only a few are active at once, the directions, or *features*, provide candidate patterns to investigate. Sparsity alone does not guarantee that a feature has one interpretable meaning. We apply it to the change that benign fine-tuning made to the model's internal representations, and test two things about the features that best separate malicious from benign days: where they activate, and whether editing them changes the anomaly score.

4.2 Hidden States and the Adaptation Difference

The model processes text through a stack of 36 layers. After each layer, each token t is represented by a *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ with $d = 4096$. Running the same text through the

model before fine-tuning (the *base* model) and after it (the *adapted* model) gives two hidden states for every token. Their difference,

$$\boldsymbol{\delta}_t \;=\; \mathbf{h}_t^{\text{ada}} - \mathbf{h}_t^{\text{base}} \;\in\; \mathbb{R}^d, \tag{4.1}$$

is the net difference in token t's representation between the two model runs. We study the hidden states after layer 26.

4.3 A Sparse Dictionary and a Worked Example

Recall the PCA reconstruction of Chapter II: a centered observation is projected onto $q < p$ orthonormal directions and reconstructed by their weighted sum. A sparse autoencoder (SAE) also minimizes reconstruction error, but uses an overcomplete dictionary of $m > d$ directions and activates at most k directions for each token [19].

Let $\boldsymbol{\mu}$ and the positive coordinate scales $\boldsymbol{\sigma}$ be estimated from the dictionary's fitting data, and put $S = \operatorname{diag}(\boldsymbol{\sigma})$. The implementation first standardizes each difference vector and then encodes it:

$$\begin{aligned} \widetilde{\boldsymbol{\delta}}_t &= S^{-1}(\boldsymbol{\delta}_t - \boldsymbol{\mu}), \\ \mathbf{z}_t &= \operatorname{TopK}_k\Big(\operatorname{ReLU}(W_{\text{enc}}\widetilde{\boldsymbol{\delta}}_t + \mathbf{b}_{\text{enc}})\Big). \end{aligned} \tag{4.2}$$

ReLU sets negative encoder outputs to zero, and TopK retains the k largest entries. Some of those entries may already be zero, so the code has at most k nonzero values. The reconstruction in the model's original coordinates is

$$\widehat{\boldsymbol{\delta}}_t = \boldsymbol{\mu} + SW_{\text{dec}}\mathbf{z}_t = \boldsymbol{\mu} + S\sum_f z_{f,t}\mathbf{w}_f, \tag{4.3}$$

where $\mathbf{w}_f$ is decoder column f. Training minimizes squared reconstruction error in standardized coordinates. The decoder has no additional learned bias.

Here $d = 4096$, $m = 8192$ and $k = 4$. The corrected dictionary is fitted for 20 epochs on two million sampled token differences at layer 26. The reconstruction objective has no label term, but the sampling procedure is label-aware: it preserves the 171,917 token rows from discovery-positive days and samples the remaining rows from the rest of the evaluation population. Confirmation-user inputs can therefore enter dictionary fitting, although their labels do not prioritize that sampling or enter feature ranking. This is a *transductive* dictionary, not a dictionary fitted on an independent training population. The low-dimensional object is each token's sparse code; the dictionary as a whole has more directions than the original space.

For an illustration, take $d = 2$, a dictionary of $m = 3$ unit directions $\mathbf{w}_1 = (1,\ 0)$, $\mathbf{w}_2 = (0,\ 1)$ and $\mathbf{w}_3 = (0.6,\ 0.8)$, one active feature ($k = 1$), $\boldsymbol{\mu} = \mathbf{b}_{\text{enc}} = \mathbf{0}$, $S = I$, and $W_{\text{enc}} = W_{\text{dec}}^{\top}$. For $\boldsymbol{\delta} = (1.2,\ 1.6)$ the scores are $\mathbf{a} = (1.2,\ 1.6,\ 2.0)$, so feature 3 is kept and $\widehat{\boldsymbol{\delta}} = 2.0\,\mathbf{w}_3 = (1.2,\ 1.6)$ with zero error. With only the two axes available, the best single direction would leave an error of $1.2^2 = 1.44$. A different token, $\boldsymbol{\delta}' = (1.0,\ 0.1)$, is described by feature 1 with error 0.01: each token uses the few features that fit it best, which is *sparse coding*.

4.4 Choosing the Features to Test

The 70 insiders are divided into 35 discovery and 35 confirmation users. Feature selection uses corrected discovery labels and excludes the confirmation users. For each feature f, the ranking statistic is

$$g_f = \bar{z}_f^{(+)} - \bar{z}_f^{(-)}, \tag{4.4}$$

the difference between its mean token activation in the positive and negative discovery populations. The selected set $\mathcal{F}$ contains the five largest gaps. The active control set $\mathcal{C}$ uses five other features with gaps near zero and nonzero activation on at least 0.2% of ranking tokens. This activity threshold does not match either the activation frequency or the physical size

of the selected edit.

The corrected selected IDs are 3843, 3230, 3199, 3975 and 1394. IDs identify columns of this particular fitted dictionary; they are not named concepts and cannot be compared directly with IDs from another fit. The confirmation group excludes users from feature ranking, but those users were inspected earlier in the research and enter unlabelled dictionary fitting. It is therefore a check across ranking groups, not an independent confirmation set.

4.5 Where Features Activate

For a feature f and a token class c, the share of the feature's total activation that lands on class-c tokens is

$$\rho_f(c) \;=\; \frac{\sum_{t\in\mathcal{T}_c} z_{f,t}}{\sum_{t\in\mathcal{T}} z_{f,t}}, \tag{4.5}$$

and its *enrichment* compares that share with the class's share of the tokens,

$$E_f(c) \;=\; \frac{\rho_f(c)}{N_c/N}.$$

When the denominator is positive, an enrichment of 1 means activation mass is proportional to token frequency; it does not measure how often the feature fires. Larger values mean more activation mass per token on class c, up to the maximum $1/(N_c/N)$ reached when all of a feature's activation lands on that class.

If a feature has zero total activation in a group, its location fraction is undefined. Table 4.1 reports session-token mass and the number of confirmation users on whose incident-window days the feature activates at least once. Four features place virtually all of their incident-window activation mass on session tokens. Feature 3843 has a mixed location and much weaker confirmation activity. Location answers where a feature activates. It does not identify all information carried there: a session state can depend on the profile text preceding it. Nor does location alone show which predictions would change if that feature were edited.

Table 4.1: Corrected r4.2 attribution. The discovery group has 695 incident-window days from 35 users; confirmation has 667 days from 35 users. Percentages pool activation mass within each group. Values displayed as 100.00% are rounded.

Feature	discovery SES (%)	confirmation SES (%)	active users
3843	91.45	84.95	13/35
3230	100.00	100.00	32/35
3199	100.00	100.00	24/35
3975	100.00	100.00	30/35
1394	100.00	100.00	30/35

4.6 Causal Test: Removing Features and Re-Scoring

Attribution is descriptive. To test whether the selected representation contributes to prediction, remove its decoded contribution from the adapted hidden state:

$$\mathbf{h}'_t = \mathbf{h}^{\text{ada}}_t - \alpha S \sum_{f \in \mathcal{F}} z_{f,t} \mathbf{w}_f, \qquad 0 \le \alpha \le 1. \tag{4.6}$$

This leaves the original hidden state and its reconstruction residual intact except for the specified subtraction. At $\alpha = 0$ it is an exact no-op; at $\alpha = 1$ it removes the selected decoded contribution. Tokens with no selected activation remain unchanged. Re-encoding the edited state need not reproduce the intended sparse code, so the experiment is defined by this physical subtraction, not by assuming independent feature coordinates.

The remaining model layers predict the same target tokens. For a day D, let $\Delta_{\mathcal{F}}(D) = \bar{\ell}'_B(D) - \bar{\ell}_B(D)$ be the change in behavior-target loss. A positive value means prediction became worse. Two controls answer different questions: removal of the active-control features tests a separate feature-selection policy, while a random dictionary direction at the same edited positions is matched to the realized per-token edit norm within 5%. Matching is checked after numerical rounding in the model's precision. The fixed feasibility policy for those dictionary controls is retained from the corrected recovery protocol; it is not a new, independently preregistered test. The active-feature control is not dose matched.

The primary removal strength is fixed at $\alpha = 1$. For each incident-window receiver, the experiment also uses a same-user window-negative day, chosen by nearest calendar day, preferring the same weekday and using fixed hash tie-breaking. Negative days can be reused. For a receiver pair (D^+, D^-), the relative window effect is

$$\begin{aligned}\tau(D^+, D^-) = [\Delta_{\mathcal{F}}(D^+) - \Delta_{\mathcal{C}}(D^+)] \\ - [\Delta_{\mathcal{F}}(D^-) - \Delta_{\mathcal{C}}(D^-)].\end{aligned} \tag{4.7}$$

A positive value would indicate a larger selected-versus-active response inside the incident window. Pairing is observational: even a positive value would not by itself prove an attack-specific circuit. Days are averaged within each user, then users are weighted equally; zero edits and repeated negative-day appearances retain their prescribed weights.

4.7 Corrected Mechanistic Results

The corrected run covers all 1,362 incident-window days and 380 unique matched window-negative days, giving 1,742 receivers and 13,936 scored rows across eight conditions. The primary summary uses the 667 incident-window days of the 35 confirmation users. Zero-edit and implementation checks passed, and the aggregate estimates were independently reproduced.

Table 4.2: Behavior-target loss changes at full removal strength on the 35 confirmation users, in nats per token. Intervals are descriptive, unadjusted 95% intervals from 10,000 user-bootstrap draws. The final row is the incident-minus-window-negative contrast in (4.7); the other rows describe incident-window receivers.

Contrast	mean	95% interval
Selected minus baseline	0.004411	[0.001812, 0.007283]
Selected minus matched direction	0.005531	[0.002886, 0.008625]
Selected minus active control	0.004275	[0.001709, 0.007179]
Incident minus window-negative	−0.004599	[−0.009430, −0.000616]

Removing the selected contribution raises behavior loss relative to baseline and to both

specified controls. This supports a contribution to prediction under this edit. However, the relative response is smaller on incident-window days than on their matched window-negative days: the final contrast is negative. The results therefore do not support an enhanced attack-specific response under this policy.

An intervention's effect on prediction loss is also different from an improvement in anomaly detection. These estimates do not demonstrate better ranking, semantic identification of the features, or mediation of the profile's influence. Together with Chapter III, they support two bounded conclusions: score composition affects insider ranking, and the selected sparse directions contribute to behavior prediction. The second does not explain the first by itself.

4.8 Limitations and Conclusions

The analysis uses one adapted model and one synthetic benchmark release. Model-seed variation and transfer to real organizational data remain untested here. The layer and dictionary settings were informed by earlier experiments; the corrected dictionary is transductive, its sample prioritizes discovery-positive rows, and its confirmation users were previously inspected. Feature ranking excludes those users, but that separation does not make the entire analysis independent.

The controls have distinct scopes. Active-feature removal compares selection policies with different natural edit sizes. Dictionary directions match edited positions and realized norms, but do not enumerate every possible alternative intervention. Same-user negative days control some user differences but are not verified benign ground truth. The reported intervals are conditional on the fixed model, dictionary, feature sets, pairs and controls.

Finally, an activation at a behavior-token position need not represent behavior alone. Interpretable labels for individual features require further evidence, and these joint edits do not identify a complete computational circuit. The corrected ablation is the causal test

reported in this thesis; the earlier donor-search result is not carried forward as current evidence after label correction.

REFERENCES

Bibliography

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11), 559–572.

[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.

[3] Jolliffe, I. T. (1986). *Principal Component Analysis*. Springer Series in Statistics. Springer, New York. https://doi.org/10.1007/978-1-4757-1904-8

[4] Software Engineering Institute. (2016). *Insider Threat Test Dataset*. Carnegie Mellon University Software Engineering Institute. https://doi.org/10.1184/R1/12841247.v1

[5] Glasser, J., & Lindauer, B. (2013). Bridging the gap: A pragmatic approach to generating insider threat data. In *2013 IEEE Security and Privacy Workshops* (pp. 98–104). IEEE. https://doi.org/10.1109/SPW.2013.37

[6] Le, D. C., Zincir-Heywood, N., & Heywood, M. I. (2020). Analyzing data granularity levels for insider threat detection using machine learning. *IEEE Transactions on Network and Service Management*, 17(1), 30–44. https://doi.org/10.1109/TNSM.2020.2967721

[7] Shlens, J. (2014). *A Tutorial on Principal Component Analysis*. arXiv. https://doi.org/10.48550/arXiv.1404.1100

[8] Cappelli, D., Moore, A., & Trzeciak, R. (2012). *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes (Theft, Sabotage, Fraud)*. Addison-Wesley Professional.

[9] HPC Wiki, The University of Southern Mississippi. (2024). *Main Page*. Magnolia HPC Wiki. Suggested infrastructure acknowledgment: The authors acknowledge HPC at The University of Southern Mississippi supported by the National Science Foundation under the Major Research Instrumentation (MRI) program via Grant # ACI 1626217. `http://magnolia.usm.edu/index.php?title=Main_Page&oldid=1116`

[10] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.

[11] Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., & Robinson, S. (2017). Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. In *AAAI Workshop on Artificial Intelligence for Cyber Security*.

[12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.

[13] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[14] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*.

[15] Qwen Team. (2025). *Qwen3 technical report*. arXiv preprint arXiv:2505.09388.

[16] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N. L., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*. `https://transformer-circuits.pub/2023/monosemantic-features`

[17] Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models. arXiv preprint arXiv:2309.08600.

[18] Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.

[19] Gao, L., Dupré la Tour, T., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024). Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093.